

Detecting and Classifying Brain Tumors from MRI Scans with Machine Learning

Isaac Braun
School of Computing
Southern Adventist University
Collegedale, Tennessee 37315
Email: isaacbraun@southern.edu
Telephone: (419) 706-0480

Abstract—In the field of medicine, there is always room for technological advances and improvements that increase diagnosis accuracy and ease medical professionals’ jobs. This paper applies machine learning to detect and classify brain tumors in MRI scan imagery. Extensive and multiple approaches in engineering the data and training convolutional neural networks were explored, with varying effects and results. This experimentation produced valuable insights about how to handle data in this domain of work, what parameters work well, and a solid predictive model boasting a 97.6 accuracy score.

I. INTRODUCTION

Even with so many advances in the field of medicine, doctors and other medical professionals are still overworked. This causes patients to wait long periods between the time of a scan or test and when they receive the results. Not only does this cause stress and frustration among patients, but the strain on medical professionals can cause inaccurate diagnoses. Assisted by automated classifications and decisions, the time required to decipher a scan or test can be reduced, allowing the professionals to use their irreplaceable knowledge and judgment in those crucial final decisions. This would reduce professionals’ working and mental load, making room for clearer and quicker judgments. As a result, as machine learning technology is implemented in the field, both waiting times and inaccurate diagnoses can be minimized.

This paper explores the possibility of applying this concept to magnetic resonance imaging results, referred to as MRIs, in order to give professionals an initial prediction of the scan result. This result may be either the lack of a tumor or the presence of one of two tumor types: Pituitary and Meningioma. In the process of engineering and manipulating the data to achieve significant results, a third class of tumor, Glioma, was removed from the dataset. VGG and Inception topologies were explored throughout the training process, and numerous layer setups, optimizers, and training options were tested. This resulted in promising model results and valuable insights about model parameters and data engineering in the application of MRI brain tumor scans.

II. DATASET AND PROCESSING

A. Source & Description

The dataset was sourced via Kaggle.com [1], where Bala Mosay uploaded over 3,200 MRI brain scan images. The

TABLE I
NUMBER OF SAMPLES IN EACH CLASS

Split	Class	Count
Training	No Tumor	395
	Glioma Tumor	826
	Meningioma Tumor	822
	Pituitary Tumor	827
Testing	No Tumor	105
	Glioma Tumor	100
	Meningioma Tumor	115
	Pituitary Tumor	74
Total		3,264

samples were provided as grayscale in JPG format, featuring either a front-facing or side view of a patient’s brain. In addition to the no-tumor class of images, there are three classes where, in each sample, a tumor mass can be clearly seen. See Figures 1 and 2 for examples of Pituitary samples. These classes were already split into training and testing sections, making working with the data especially easy. Table I displays the number of images found in each class.

A Python [2] script utilizing TensorFlow [3] through Keras [4] was used for this project, and here, Keras utilities were used to read and manipulate the data. This made the process for getting the images into the models smooth and quick. The `image_dataset_from_directory` method was used to read in both the training and testing images in the way that Keras models accept them, along with resizing the images to a unified size and applying other manipulation parameters.

B. Data Engineering

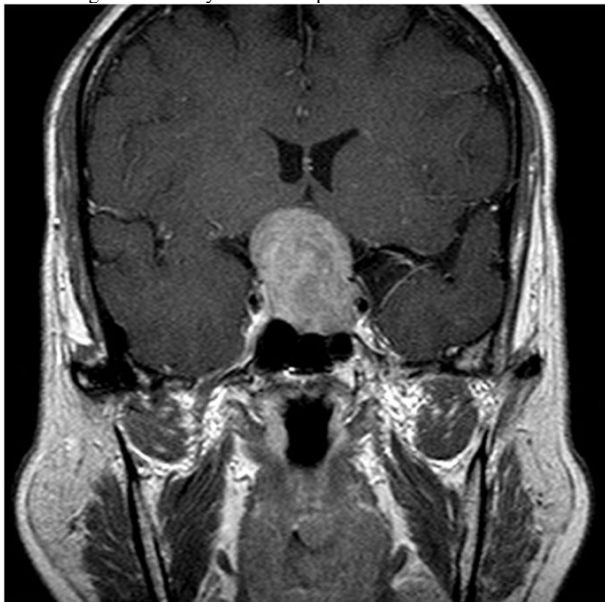
After experimentation, only a few data engineering steps proved to be needed. Initially, all four classes were used and were prepared in three steps:

- 1) Many images in the tumor classes were around 512 by 512 pixels but with some variation. The no-tumor class had images varying around 200 pixels in height and width. Considering this variance, all the images were resized to 224 by 224 pixels to create consistency for the models’ training. This resizing was done by passing the desired height and width

Fig. 1. Pituitary tumor sample viewed from the side



Fig. 2. Pituitary tumor sample viewed from the front



to `image_dataset_from_directory` through the `image_size` parameter.

- 2) As the MRI images are in grayscale, initially, the images were reduced to only one grayscale channel with `image_dataset_from_directory` using the `colormode` parameter. Since this proved to have no effect and the pre-trained VGG and Inception models only accept 3 channel inputs, this step was eliminated.
- 3) As is standard practice to help speed up and normalize training, the image pixels were rescaled to be within a range of zero to one. This was done as the first layer in the models' through the Keras rescaling utility layer:

`Rescaling(1./255).`

III. RESULTS

A. Initial Experiments & Topologies

Multiple topologies were explored in the process of finding a suitable and capable model. The first one explored was a basic one-block VGG model, which is a sequential (`Sequential()`) model with one convolutional layer (`Conv2d()`), one max pooling layer (`MaxPooling2D()`), one flattening layer (`Flatten()`), and one dense layer (`Dense()`). The Keras code for this model can be seen in Figure 3. This topology produced unsatisfactory results, with an accuracy of around 50 percent, so a two-block VGG model was tested.

The two-block VGG model is very similar to the one-block but adds an additional convolutional layer and max pooling layer. These layers come after the first "block" of convolutional and max pooling layers and double the number of input filters in the convolutional layer to allow for more connections and room to train. Figure 4 shows a slightly modified version of this topology that includes the later discussed dropout layer. This topology produced better results, with a testing accuracy of around 65 or 70 percent, which was still unacceptable for the desired use case. Therefore, a three-block model was explored.

Again, the three-block VGG model adds another block of convolutional and max pooling layers with doubled input filters. See Figure 5 for the Keras implementation. Like the two-block model, this model increased the accuracy slightly but stayed around the 70 percent mark, which was not good enough. This led to the exploration of transfer learning, where the last few layers of a model pre-trained on massive amounts of data are modified to train on a new dataset. Two pre-trained models, VGG16 and InceptionV3, were tried for this experiment. These are both models made available through the Keras library, and a short bit of Python code was used to modify the last layers to use the MRI dataset and predict one of the classes (see Figure 6 for reference). Unfortunately, neither of these models produced much better or more promising results than the custom VGG models.

B. Combating Overfitting

During the testing of all model variations, the prediction accuracy on the training data was around 90 to 100 percent, which is very good. Still, the prediction accuracy on the testing (sometimes called validation) data never rose above 75 percent or above the training data accuracy. While a 70 or so percent testing accuracy may be high enough to be considered a relatively good model, the large gap between the testing and training accuracy is entirely unacceptable. This behavior of higher training accuracy than testing accuracy is called overfitting and should generally be avoided.

Overfitting expresses that a model is more accurate at predicting the class of a data sample it was trained on than a data sample it has never seen before. A model with significant amounts of overfitting will more than likely not perform well

Fig. 3. One Block VGG topology using Keras

```
model = Sequential([
    Input(input_shape),
    # Rescale images to [0, 1] range
    Rescaling(1./255),
    Conv2D(
        32, (3, 3),
        activation='relu',
        kernel_initializer='he_uniform',
        padding='same'
    ),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(num_classes, activation=dense_activation)
])
```

in real-world applications, as it will struggle to make accurate predictions with new and unseen data. Due to this constant overfitting, experimentation with data manipulation was explored with the hope of reducing the overfitting. Conceptually, if a model is trained on data with more variance, it will have more data points to understand the data better and be able to make more correct predictions. Therefore, augmenting the data can create more variance and, hopefully, a more accurate model. Primarily, three methods were attempted, but two of the three proved to be more harmful than helpful.

- 1) Random Image Rotation: A common technique used to combat overfitting, this action was executed using the Keras `RandomRotation` layer in the models' topology. This did not decrease the overfitting problem but decreased the models' performance rather than improving it. After a little contemplation, it was realized that the nature of the data was a likely culprit for this outcome. As the different classes of tumors commonly appear in similar areas of the scans, rotating the image would remove a major decision point for the models.
- 2) Random Image Flipping: Another common technique implemented through the Keras `RandomFlip` layer in the model's topology. Like random image rotation, this method proved unfruitful in helping the overfitting problem. The reasoning is possibly due to the exact reason for random image rotation: tumor position is a crucial point of data.
- 3) Dropout: This approach is when a portion of neurons in the model are dropped to reduce the complexity of the models' decision process. It is very common in VGG topologies to include a dropout layer after the max pooling layer in each block with a small amount at the end of each block and half of the neurons right before the last dense layer. This was done using the Keras `Dropout()` layer and can be seen in Figures 4 and 5.

As previously mentioned, all four classes were used in the initial experiments. These models were trained in a wide range of epochs, anywhere from 10 to 100, to try and find the best result as well as to see the slope of the accuracy graph during training using a graph like the one seen in Figure 7. Looking

Fig. 4. Two Block VGG topology using Keras

```
model = Sequential([
    Rescaling(1./255),
    Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.2),
    Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(num_classes, activation=dense_activation)
])
```

Fig. 5. Three Block VGG topology using Keras

```
model = Sequential([
    Input(shape=input_shape),
    Rescaling(1./255),
    Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.1),
    Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.1),
    Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_uniform', padding='same'),
    MaxPooling2D((2, 2)),
    Dropout(0.1),
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.1),
    Dense(num_classes, activation=dense_activation)
])
```

at the trend of the graph line throughout the epochs, it is easier to see if a model will continue to improve with more training time or if it has stalled out. This is also useful to find peaks where the model has a good testing accuracy and decide to stop training earlier at that point.

C. Changing Number of Classes

After failing to produce a capable model without massive amounts of overfitting, the attention was turned to the classes themselves. With hopes of reducing the complexity of what the image was required to learn, the models were trained using three classes and only two classes. Multiple variations were tested by swapping out classes to see if a better result could be achieved.

The two-class models were first trained with the no-tumor and the Glioma classes and produced results that were not much better than those trained with four classes. Even though this model would be practically useless, as a medical professional could easily see if there was a tumor in an MRI image, this result was quite confusing. With MRI images of tumor-free brains in one class and images of brains with tumors in the other, it would be understandable to think that the model should have had an easier time learning and predicting accurate results.

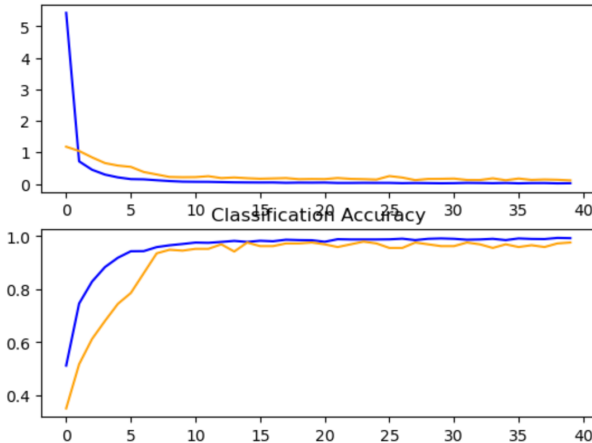
The other two-class models were trained using the Pituitary and Meningioma classes instead of the Glioma class. While these results were respectable, the practicality and usability of these models would not be very high. Medical professionals would more than likely be slowed down by using this model to predict if a tumor that belongs to a specific class was present in the MRI rather than sped up. While models trained purely to detect tumors can be extremely helpful in validation and

Fig. 6. VGG Transfer Learning topology using Keras

```
base_model = VGG16(weights="imagenet", include_top=False, input_shape=input_shape)
base_model.trainable = False ## Not trainable weights

model = Sequential([
    base_model,
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.4),
    Dense(num_classes, activation='softmax')
])
```

Fig. 7. Loss and accuracy graph from the produced model



finding small masses that a medical professional may miss, this experimentation was not aimed at that.

The three-class models were tested in a similar manner by rotating classes to find the best results. The first tests were done with the no-tumor and Glioma classes, with either the Pituitary or Meningioma class as the third. These models resulted in the same or worse results, with testing accuracy barely reaching 60 or 70 percent accuracy before the training accuracy reached the high 90 percent range. When evaluating the accuracy graph of these models, it could be seen that there was a plateau and a very unlikely chance of the testing accuracy ever increasing with more training.

After the first three-class model tests using the Glioma class, another test was done with the no-tumor, Pituitary, and Meningioma classes. This model immediately performed significantly better than any other model created before. With even the most basic one-block topology, it produced testing accuracy scores of around 85 percent and training accuracy scores nearing 100 percent. Better yet, the graph showed an upward trend, which gave hope that the accuracy would improve with more epochs or an adjusted topology, and a better model could be produced.

After this breakthrough, more research was done on the tumor classes, and it was found that the Glioma class is a looser classification of tumors that may not all have the same position, shape, or size. This correlates with the models' struggles and makes sense that the model would struggle to learn accurately when one of the classes has a very high variance. With this new knowledge, all model topologies were

trained and tested again. Based on the output of the accuracy graph, some models were trained for differing amounts of epochs.

- 1) The one-block model produced a solid accuracy result of around 95 percent, but the graph did not show promise of continual growth, so it was not explored further.
- 2) The two-block model produced a solid result of around 96 percent accuracy and showed a climbing line on the graph, indicating the potential for more growth.
- 3) Similarly to the one-block model, the three-block model produced results of around 95 percent accuracy but showed little indication of growth on the graph.
- 4) Both transfer learning models produced slightly lower accuracy scores in the 92-93 percent range but were more overfitted and had a clearer plateau in the graphs.

After seeing these results, more effort was put into refining the two-block model. It was run for 100 epochs multiple times while adjusting the dropout amount and the optimizer learning rate. Eventually, a peak was spotted around epoch 40, so the early stopping principle was utilized. When run at 40 epochs, the model produced a 97.62 percent testing accuracy and a 99.34 training accuracy. Refer to Figure 7 for the full epoch breakdown and loss values.

While this model is still overfitted, it has such a high accuracy rate that it should be capable of handling new data quite well. In order to verify this, 61 images were chosen randomly from the testing dataset and evaluated with this new model. For these 61 images that are unknown to the model, it predicted all but 1 of them correctly. While not perfect, this result is enough to be useful in a real-world application.

IV. CONCLUSION

After much experimentation and struggle, a two-block VGG topology model was produced that can classify MRI images of the brain into one of three classes: no-tumor, Pituitary, and Meningioma. While slightly overfitted, this model has a very high accuracy of 97.62 percent. Not only was a well-performing model produced from this experiment, but a few observations for working with image classification models were made.

- 1) If it is not possible to have a person knowledgeable in the data domain on the team, do research into the data to understand it as well as possible. In this experiment, if it were understood that the images should not be manipulated and that the Glioma class was very broad, it would have greatly simplified and shortened the process.
- 2) Adjusting the level of dropout can have a great effect and so should be tested with differing amounts.
- 3) Train for higher epochs more frequently when a graph shows a good indication of growth.

A. Future Work

The future is bright for this experiment. The suggested next steps would be to do more research into the data, maybe by bringing a medical professional into the team, and gather more data to train the model on. This would hopefully increase

the accuracy and robustness of the model, as well as open up the possibility of bringing the Glioma class back into the data. Lastly, this new model could be applied to an existing or new system that would reduce the workload of medical professionals in their daily work.

REFERENCES

- [1] B. Mosay, "Brain tumor dataset." [Online]. Available: <https://www.kaggle.com/datasets/balamosay/brain-tumor-dataset?resource=download>
- [2] Python. [Online]. Available: <https://www.python.org/>
- [3] TensorFlow. [Online]. Available: <https://www.tensorflow.org/>
- [4] Keras.io. [Online]. Available: <https://www.keras.io/>